

Guia Prático de Deploy e Testes para a Equipe (UD-H1 Navigation)

Este guia foi elaborado para demonstrar que **não há mudanças na rotina operacional da equipe**. O fluxo de trabalho de compilação, gravação do Arduino e execução do robô permanece **exatamente o mesmo** que vocês já utilizavam, pois as modificações de correção foram aplicadas diretamente nos arquivos internos de configuração.

1. Como Gravar o Arduino (Sem Alterações)

O firmware do Arduino foi restaurado para a versão estável e original com a qual vocês já trabalhavam.

1. Conecte o cabo USB do Arduino ao seu computador ou Jetson.
2. Abra a **Arduino IDE**.
3. Abra o arquivo localizado no repositório em: [odometrypulse-malha.ino](#)
4. Selecione a placa correta (ex: *Arduino Uno* ou *Mega*) e a respectiva porta serial.
5. Clique em **Carregar** (*Upload*).

[!NOTE] Como o firmware foi revertido ao original, o comportamento mecânico inicial e o controle de TRIM são exatamente os mesmos anteriores.

2. Como Compilar o Workspace (Sem Alterações)

Para garantir que o ROS 2 registre as modificações e os novos nós auxiliares (**safe_stop** e **wasd_teleop**), compile o workspace local da mesma forma:

```
# 1. Navegue até a pasta raiz do repositório (Navigation_Unip_Droidians)
cd ~/athome_ws # (ou o seu caminho de workspace)

# 2. Compile os pacotes
colcon build --symlink-install

# 3. Carregue as variáveis de ambiente do workspace
```

```
source install/setup.bash
```

[!TIP] O parâmetro `--symlink-install` garante que qualquer alteração subsequente que vocês façam nos scripts Python (como os drivers ou nós de teleoperação) seja aplicada **imediatamente** sem precisar compilar o workspace novamente.

3. Como Rodar a Navegação (Sem Alterações)

Para executar todo o sistema de navegação física (leitura do LIDAR, odometria do Arduino, filtro do laser e a pilha do Nav2), o comando permanece idêntico:

```
ros2 launch my_robot_bringup navigation.launch.py
```

O que acontece por trás dos panos agora?

- O ROS 2 carrega o driver serial original (`base_driver.py`).
- O ROS 2 inicializa automaticamente o nó de segurança `safe_stop` (que protege o robô contra colisões frontais a menos de 30 cm).
- O Nav2 lê o arquivo de parâmetros `nav2_params.yaml` sintonizado.
- **O erro de travamento foi resolvido aqui:** O Nav2 lê que a velocidade mínima lateral exigida (`min_y_velocity_threshold`) é `0.001` m/s (antes estava em `0.5` m/s). Isso permite que o Nav2 envie comandos lineares e angulares sem achar que o robô está travado no eixo Y.

4. Como Testar e Validar na Prática

Com o launch de navegação rodando, vocês podem validar o funcionamento do robô de duas formas:

Método A: Teste por Teleoperação WASD (Garantia de Direção)

Em um novo terminal (lembrando de rodar `source install/setup.bash` nele), digite:

```
ros2 run serial_com_py wasd_teleop
```

- Use as teclas `W`, `A`, `S`, `D` para controlar o robô.
- Pressione `Espaço` para frear.

- Aproxime o robô de uma parede para a frente e verifique se o nó de segurança (**safe_stop**) bloqueia o avanço para evitar colisões.

Método B: Teste Autônomo via RViz2 (Navegação com Nav2)

1. No painel do **RViz2** (que abre automaticamente junto com o comando do item 3):
2. Clique no botão "**2D Pose Estimate**" no topo da tela e indique a posição/direção aproximada do robô no mapa.
3. Clique no botão "**Nav2 Goal**" e selecione um ponto de destino no mapa.
4. O robô irá traçar e seguir a rota de forma contínua, sem sofrer os cancelamentos ou congelamentos causados pelos limites incorretos de velocidade e tempo da configuração antiga.